BetaZero: MCTS With Learned Backup

Design Document

David Shin

April 22, 2026

Contents

1 Motivation **2**

2 Related Work **2**

3 Background **3**
3.1 NNUE . . . 3
3.2 KataGo's Uncertainty Head . . . 3

4 Architecture Overview **4**
4.1 Network Structure . . . 4
4.2 Latent Projection ($\mathbf{z}$) . . . 5
4.3 Backup Network . . . 5

5 Training Targets **6**
5.1 Targets for Q and W . . . 6
5.2 Targets for AV and AU . . . 7

6 Training Procedure **8**
6.1 Sampling for $\mathcal{A}_\phi$ . . . 8
6.2 Bootstrapping and Regularization . . . 8

7 Inference and Search Integration **8**
7.1 NNUE-Based Backup . . . 8
7.2 Use in PUCT . . . 9
7.3 Future Direction: Replacing PUCT . . . 9

8 Spatial Latent Representations **9**

9 Hyperparameters **10**

1. Motivation

AlphaZero aggregates child value estimates into a parent estimate using a visit-count-weighted average:

$$Q(p) = \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i}.$$

This is a reasonable heuristic but has well-known theoretical weaknesses. It conflates *exploratory* visits (induced by PUCT's exploration term and auxiliary mechanisms like policy temperature and Dirichlet noise) with *confirmatory* visits (made because a child is believed to be good). A child that was heavily explored but ultimately found to be poor will still exert downward pressure on $Q(p)$ proportional to its visit count. Additionally, a child suddenly found to be very good needs many more visits to offset highly-visited-but-inferior siblings than should be theoretically necessary.

Experienced go and chess players intuitively understand the suboptimality of estimating a parent node as an average over its children. Game theoretically, the value of a position is determined by the opponent's **best** response, not by an average over possible responses.

BetaZero replaces this hand-crafted backup with a **learned backup network** $\mathcal{A}_\phi$ that takes children statistics as input and outputs a posterior parent estimate $Q(p)$. The hope is that $\mathcal{A}_\phi$ learns to discount exploratory visits and produce better-calibrated parent estimates, which in turn improves the quality of search.

2. Related Work

Guez et al. (2018) proposed MCTSnets[1], an architecture that replaces **all** components of MCTS — selection, evaluation, and backup — with learned neural network functions, trained end-to-end via gradient-based optimization. Each node in the search tree stores a learned vector-valued memory, and a backup network updates parent memory based on child memory along the traversed simulation path. MCTSnets was demonstrated on Sokoban with 25 simulations, significantly outperforming standard MCTS baselines.

While MCTSnets validates the idea that learned backup can outperform hand-crafted backup, its fully end-to-end design has two significant drawbacks. First, replacing selection, evaluation, and backup simultaneously provides too many degrees of freedom and too weak a learning signal to scale practically. The training signal (task reward) must drive learning across all components at once, with no intermediate supervision. The vector-valued node statistics are opaque, making the system difficult to reason about, debug, or regularize. Second, MCTSnets suffers from a computational issue: even if the backup network could be well trained, evaluating a neural network at every backup step makes the overall system impractical to use in competitive settings, where millions of backup steps are performed per move.

BetaZero can be viewed as a more constrained version of this idea. It retains AlphaZero's selection (PUCT) and evaluation (value network) intact, and focuses specifically on improv-

[1] A. Guez, T. Weber, I. Antonoglou, K. Simonyan, O. Vinyals, D. Wierstra, R. Munos, D. Silver. *Learning to Search with MCTSnets.* ICML 2018. `https://arxiv.org/abs/1802.04697`

ing the backup step. The backup network's outputs are interpretable scalars (Q and W) with well-defined training targets (game outcomes and uncertainty estimates). This allows for more precise learning with intermediate supervision, rather than relying on end-to-end credit assignment through the entire search computation graph. The computational issue is addressed by adopting NNUE mechanics (see Section 3.1).

3. Background

3.1. NNUE

The Efficiently Updatable Neural Network (NNUE)[2] was introduced by Yu Nasu in 2018 for use in computer shogi engines. The key insight is that when a neural network's input changes only sparsely between evaluations, the first layer's output can be maintained incrementally rather than recomputed from scratch.

In the standard NNUE design, the first layer maps a large, sparse input (encoding the board position) to a hidden representation called the **accumulator**. When a move is made, only a few input features change (a piece leaves one square and arrives at another). Rather than recomputing the full first-layer matrix-vector product, the accumulator is updated via a **subtract-add** operation: subtract the weight columns corresponding to removed features and add the columns corresponding to added features. The subsequent layers are small and recomputed from scratch using the updated accumulator. This makes the per-evaluation cost independent of the total input dimensionality, depending only on the number of changed features.

NNUE requires very specific structural conditions: a large, sparse first layer whose inputs change incrementally, followed by small dense layers. These conditions have limited its adoption to a narrow set of games; it has been successfully applied to shogi and chess, but has not been adapted to go. As we shall see, BetaZero's backup network has been carefully architected to satisfy these conditions.

3.2. KataGo's Uncertainty Head

KataGo[3] introduces an **uncertainty head** U alongside the standard value and policy heads. This head predicts how much the value estimate at a given position is expected to shift over the remainder of the game, trained against the target:

$$U(p)_{\text{target}} = \big(V(p) - (1-\lambda)\sum_{t \geq T} Q_t \lambda^{t-T}\big)^2,$$

where T is the current turn, Q_t is the final search Q at the root at the end of turn t, and λ is an exponential decay parameter (KataGo uses $\lambda = 5/6$). KataGo uses U to weight visits during backup: nodes with higher uncertainty receive proportionally more search effort.

[2] Y. Nasu. *Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi.* Ziosoft Computer Shogi Club, 2018. `https://www.chessprogramming.org/NNUE`

[3] D. Wu. *Accelerating Self-Play Learning in Go.* 2019. `https://arxiv.org/abs/1902.10565`. See also: `https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md#uncertainty-weighted-mcts-playouts`

BetaZero adopts this uncertainty head, but repurposes it: rather than using U to weight visits, we provide it as an input to the backup network $\mathcal{A}_\phi$, allowing the network to condition its backup strategy on the uncertainty of each child.

4. Architecture Overview

4.1. Network Structure

During training, there is a single neural network comprising a ResNet trunk, several output heads, and a backup subnetwork. Figure 1 shows the overall structure.

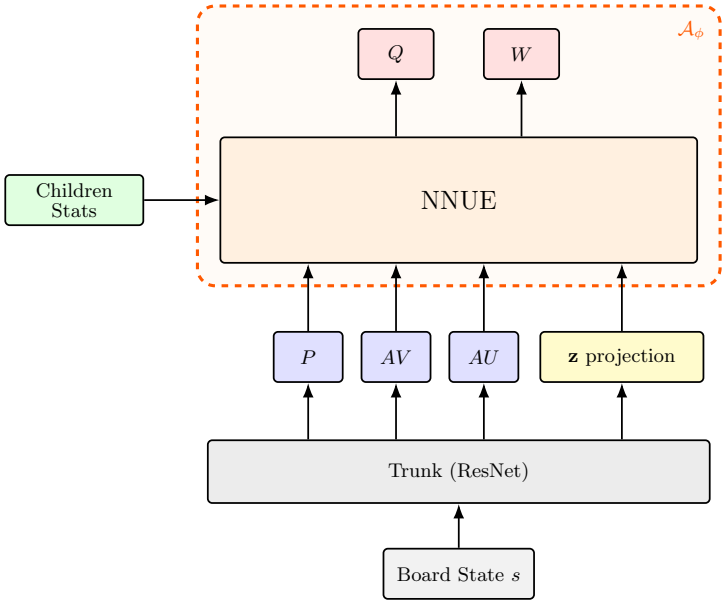

Figure 1: Network architecture. The trunk feeds into heads P, AV, AU, and a $\mathbf{z}$ projection, all of which feed into the NNUE-based backup subnetwork $\mathcal{A}_\phi$ alongside dynamic children statistics from the search tree. $\mathcal{A}_\phi$ outputs Q (posterior value) and W (posterior uncertainty). When all children statistics are zero (i.e., at node expansion before any search), Q and W reduce to the prior estimates V and U. At export time, $\mathcal{A}_\phi$ is copied out for CPU-side incremental evaluation via NNUE mechanics.

The trunk produces features that branch into:

- **Policy head** P: prior move probabilities (same as AlphaZero).
- **Action-value head** AV: per-action value predictions (see Section 5.2).
- **Action-uncertainty head** AU: per-action uncertainty predictions (see Section 5.2).
- **z projection**: a compressed representation of the trunk features (see Section 4.2).

These four outputs, together with dynamic children statistics from the search tree, feed into the backup subnetwork $\mathcal{A}_\phi$, which outputs:

- Q: posterior value estimate.

- W: posterior uncertainty estimate.

When all children have zero visits ($N_i = 0$ for all i), $\mathcal{A}_\phi$ has no search evidence to work with, and its outputs reduce to the **prior** estimates: $Q = V$ (prior value) and $W = U$ (prior uncertainty). There are no separate V or U heads; the prior and posterior estimates are produced by the same subnetwork, with the children statistics determining the regime.

At expansion time, the entire network is evaluated on the GPU in a single forward pass, with children statistics set to zero. This produces P, AV, AU, Q ($= V$), W ($= U$), and critically, the intermediate activations of $\mathcal{A}_\phi$ (the $\mathbf{z}$ vector and the NNUE accumulator state). These intermediates are stored at the node to enable incremental CPU-side evaluation on subsequent backups.

At export time, the $\mathcal{A}_\phi$ portion of the network is copied out as a separate lightweight network. On each backup, only $\mathcal{A}_\phi$ is re-evaluated on the CPU using NNUE mechanics (see Section 7.1), while the trunk and other heads are not re-run.

4.2. Latent Projection (z)

The backup network conditions on positional context from the trunk. If the full trunk features were stored at each node, the memory cost would be prohibitive. To address this, a small **projection** maps the trunk features down to a compact representation $\mathbf{z}$.

The simplest option is a **flat** projection to a vector of fixed dimension d_z (e.g., 32–64), capturing global positional properties such as tactical sharpness or game phase. An alternative is a **spatial** projection that retains board structure; see Section 8 for further discussion.

4.3. Backup Network

The backup network $\mathcal{A}_\phi$ is a lightweight network evaluated at every *backup step*. Its outputs are stored at the node and read during selection (see Section 7.2). The outputs for a node p are:

- $Q(p)$: posterior value estimate for the parent node.
- $W(p)$: posterior uncertainty estimate for the parent node.

The inputs for a node p with children $\{c_1, \ldots, c_k\}$ are:

- Per-child statistics: $\{Q_i, W_i, N_i\}$ for each child c_i. For expanded children ($N_i > 0$), Q_i and W_i are outputs of $\mathcal{A}_\phi$ previously computed at child c_i. For unexpanded children ($N_i = 0$), $Q_i = AV_i$ and $W_i = AU_i$, the per-action predictions from the main network (see Section 5.2).
- The outputs $P(p)$, $AV(p)$, $AU(p)$, and $\mathbf{z}(p)$.
- Baseline aggregates derived from the Law of Total Expectation (LoTE) and the Law of Total Variance (LoTV):

$$Q^*(p) = \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i}$$
$$W^*(p) = \frac{U(p) + (V(p) - Q^*(p))^2 + \sum_i N_i (W_i + (Q_i - Q^*(p))^2)}{1 + \sum_i N_i}$$

where $V(p)$ and $U(p)$ denote the prior outputs, i.e., $Q(p)$ and $W(p)$ evaluated at $N_i = 0$. The LoTV baseline $W^*(p)$ is motivated by interpreting W as a variance. This interpretation is loosely justified: the W target is defined as a squared difference between Q values (see Section 5.1), which matches the structure of variance computation. The interpretation need not be rigorous — $W^*(p)$ serves only as an input suggestion that $\mathcal{A}_\phi$ is free to use or ignore.

Because $\mathcal{A}_\phi$ is called at every backup step (typically $O(n \log n)$ times in a search tree with n nodes), it must be small. The latent $\mathbf{z}$ provides positional context, allowing $\mathcal{A}_\phi$ to modulate its backup strategy based on the character of the position (e.g., tactical sharpness, game phase).

5. Training Targets

5.1. Targets for Q and W

$Q(p)$ and $W(p)$ are the outputs of $\mathcal{A}_\phi$. When all children have zero visits, they reduce to the prior estimates $V(p)$ and $U(p)$ respectively. Both the prior and posterior regimes are trained against the same targets; the difference is only in what information is available to $\mathcal{A}_\phi$.

Q target. The training target for $Q(p)$ is the game outcome Z. This is the same supervision signal used in standard AlphaZero for the value head. At expansion (all $N_i = 0$), $Q(p) = V(p)$ is a prior estimate conditioned only on the board state. After search ($N_i > 0$ for some children), $Q(p)$ is a posterior estimate additionally conditioned on children statistics.

Since the network can always represent the function "output $Q^*(p)$" as a special case, it can in principle recover AlphaZero's backup. The hope is that dynamic children statistics carry additional signal about the true value.

W target. $W(p)$ predicts uncertainty: how much the value estimate is expected to swing between the current position and the end of the game. We adapt KataGo's uncertainty target (Section 3.2), with one modification.

At the prior level (all $N_i = 0$), the target at turn T is:

$$U(p)_{\text{target}} = \big(V(p) - (1 - \lambda) \sum_{t>T} Q_t \lambda^{t-1-T}\big)^2,$$

where Q_t represents the final Q at the tree root at the end of turn t, and λ is a system hyperparameter.

At the posterior level (with search evidence), the target is analogous. Denoting by $Q^{(n)}(p)$ the output of $\mathcal{A}_\phi$ after n search iterations:

$$W(p)_{\text{target}} = \big(Q^{(n)}(p) - (1-\lambda)\sum_{t>T} Q_t \lambda^{t-1-T}\big)^2.$$

Note that both sums range over $t > T$, whereas KataGo's sum ranges over $t \geq T$. The reason is that the expected difference between the current $Q^{(n)}(p)$ and Q_T depends on n, the search iteration at which the sample was obtained. Including $t = T$ would introduce an inappropriate coupling between n and $W(p)_{\text{target}}$.

The critical grounding property of the W target is that $W(p) = 0$ for any position whose game-theoretic value is fully resolved — either because it directly precedes a terminal state, or because its subtree has been exhaustively searched (e.g., proven mate, proven draw). This gives the backup network a reliable anchor: $W_i = 0$ means child c_i's value Q_i is completely trustworthy, and $W_i = \epsilon$ means it is nearly so.

As noted in Section 4.3, we interpret W as a variance to compute a LoTV baseline over children statistics. This is merely a chosen *interpretation* and is not rigorously justified. It need not be rigorously justified, as the LoTV-computed value serves merely as a *suggestion* to $\mathcal{A}_\phi$ on how to compute the actual uncertainty backup, which $\mathcal{A}_\phi$ is free to use or ignore. Whether the target definition or the LoTV baseline are useful are empirical questions; what ultimately matters is whether the choices help $\mathcal{A}_\phi$ produce better $Q(p)$ estimates.

5.2. Targets for AV and AU

The main network includes per-action heads $AV(s,a)$ and $AU(s,a)$ that predict the value and uncertainty at the child state reached by action a. Their training targets are the network's own prior evaluations at those child states:

$$AV(s,a)_{\text{target}} = V(s'),$$
$$AU(s,a)_{\text{target}} = U(s'),$$

where s' is the state reached by playing action a from s, and $V(s')$ and $U(s')$ denote the prior outputs (i.e., Q and W evaluated with all $N_i = 0$) at state s'.

To produce these targets, children of sampled root nodes are expanded during training. This can be done exhaustively (expanding all children) or by sampling a random subset and computing loss only on that subset.

AV and AU serve as the default Q_i and W_i inputs for unexpanded children when feeding into $\mathcal{A}_\phi$. This allows the backup network to reason about all children — not just those that have been visited — using the network's prior predictions as stand-ins.

Note that AlphaZeroArcade[4] already uses an AV head, whose output is used for the Q term in the PUCT formula for unvisited children.

[4] `https://github.com/shindavid/AlphaZeroArcade`

6. Training Procedure

6.1. Sampling for $\mathcal{A}_\phi$

Following KataGo's Playout Cap Randomization, some fraction of root nodes are sampled for NN training. Of these, a configurable fraction α is allocated to training the network in the **node-expansion regime** (prior heads: P, AV, AU, and Q/W with all $N_i = 0$), and the remainder $1 - \alpha$ to training the network in the **node-backup regime** (Q/W conditioned on search evidence). The split ratio α is a system hyperparameter; since the trunk and expansion-regime heads are larger and learn a harder function, $\alpha > 0.5$ is likely appropriate.

For positions allocated to the backup regime, if N is the search budget in iterations, we uniformly sample some n from $\{1, \ldots, N\}$ and snapshot the tree statistics after n iterations have been performed. This means $\mathcal{A}_\phi$ learns to produce a useful posterior estimate given any amount of search evidence, from very little (n small) to nearly the full budget ($n \approx N$).

6.2. Bootstrapping and Regularization

Early in training, $\mathcal{A}_\phi$ is uninformed, which risks destabilizing the feedback loop between search quality and training targets. To mitigate this, the network is regularized during early training to match the hand-crafted baselines:

$$Q(p)_{\text{baseline}} = Q^*(p),$$
$$W(p)_{\text{baseline}} = W^*(p).$$

This regularization is annealed away as training progresses. Because $Q^*(p)$ and $W^*(p)$ are inputs into $\mathcal{A}_\phi$, they are representable and thus constitute reachable regularization targets. The network will learn initially to simply directly output these inputs, providing some initial stability. As the annealing tapers, the network should gradually mature to produce more nuanced outputs.

An additional stability mechanism comes from the architecture itself: since the prior outputs (V and U) and posterior outputs (Q and W) are produced by the same subnetwork, the prior regime ($N_i = 0$) is always well-supervised by game outcomes. This prevents $\mathcal{A}_\phi$ from drifting arbitrarily even after the LoTE/LoTV regularization is removed.

7. Inference and Search Integration

7.1. NNUE-Based Backup

The full network is evaluated once per node expansion on the GPU. This produces P, AV, AU, and the initial Q ($= V$) and W ($= U$), along with the $\mathbf{z}$ vector and the NNUE accumulator state — all stored at the node.

On each subsequent backup, only $\mathcal{A}_\phi$ is re-evaluated on the CPU using NNUE mechanics. The first layer of $\mathcal{A}_\phi$ computes a per-child embedding that is sum-pooled into an accumulator. Between consecutive backups at a given node, typically only one child's statistics change. The

accumulator is updated via a subtract-add operation — subtracting the old child embedding and adding the new one — rather than recomputing the full sum over all children. The subsequent layers are then recomputed from the updated accumulator.

This makes the per-backup cost of $\mathcal{A}_\phi$ independent of the branching factor, which is significant for games with large action spaces. The accumulator is initialized as part of the GPU forward pass at expansion time, so no separate $O(k)$ initialization is needed on the CPU.

In self-play mode, evaluations of $\mathcal{A}_\phi$ across simultaneous games are batched, amortizing cost. In single-game mode, we may be able to adopt a similar mechanism to standard MCTS parallelization, which involves multiple search threads and virtual visits. Some details remain to be worked out.

7.2. Use in PUCT

$\mathcal{A}_\phi$'s stored Q_i output for each child c_i is used directly in the PUCT selection formula:

$$\text{PUCT}_i = Q_i + c \cdot P_i \cdot \frac{\sqrt{1 + N(p)}}{1 + N_i}.$$

7.3. Future Direction: Replacing PUCT

Later, we may add a head to $\mathcal{A}_\phi$ that outputs a score that replaces PUCT entirely. The challenge is devising a training target that captures the purpose of PUCT.

One idea for this is to define a notion of "exploration worthiness". PUCT aims to identify actions worth exploring. What makes an action worth exploring? One candidate definition is to say that an action is worth exploring if doing so has strong potential for changing our current Q estimate. This suggests using a training target that is based on the change in $Q(p)$ induced by visiting a given child. A similar alternative is to say that an action is worth exploring if *not* doing so for the remaining search budget is likely to produce a different Q estimate than an unconstrained usage of the remaining search budget. Still another possibility is to locate exploration worthiness in a likely reduction of $W(p)$, rather than an expected delta in $Q(p)$. It may be possible to produce targets corresponding to these notions.

All these options have the property that we can use an exact score of 0 for fully resolved positions (e.g., proven mate, proven draw), while a NN estimated position can be made to return a strictly positive score. This has the nice property that the selection criterion would naturally avoid re-exploring fully resolved positions, a property that standard PUCT lacks.

If something can be found to work here, it may be possible to remove one or more heads among P, AU, and AV, as they exist primarily to facilitate the PUCT criterion.

8. Spatial Latent Representations

Section 4.2 introduced the latent projection $\mathbf{z}$ as a compressed representation of the trunk features. The simplest design is a flat vector of dimension d_z, which captures global positional properties but discards spatial structure. An alternative is to retain the board geometry by

projecting to a **spatial tensor** of shape (H, W, r), where $H \times W$ matches the board dimensions and r is a small channel depth (e.g., 4–8).

A spatial $\mathbf{z}$ enables $\mathcal{A}_\phi$ to represent **correlation beliefs between actions**. For example, in Go on a 19×19 board, a spatial projection of shape $(19, 19, r)$ could learn to map each candidate move to a vector on an r-dimensional hypersphere, with the cosine similarity between two move-vectors encoding the backup network's belief about how correlated their values are. This would allow $\mathcal{A}_\phi$ to reason about questions like: "move A and move B both attack the same group, so if A's value dropped after search, B's value likely dropped too." A flat $\mathbf{z}$ cannot represent such per-action structure.

The memory cost of a spatial $\mathbf{z}$ is $(H \times W \times r)$ floats per node. For Go with $r = 8$, this is $19 \times 19 \times 8 = 2{,}888$ floats — reasonable for practical use.

This design is more natural for games with spatially structured action spaces (Go, Hex, Othello) than for games like Chess, where the action space is a set of (source, destination) pairs rather than single board intersections.

Whether spatial $\mathbf{z}$ actually helps is an empirical question. Even if the representation has the capacity to encode correlations, the backup network must learn to use them, and the training signal (game outcomes) may be too diffuse to drive fine-grained correlation learning. The flat design is the safe default; spatial $\mathbf{z}$ is an avenue for games where it is natural.

9. Hyperparameters

Table 1 summarizes the tunable hyperparameters of the system.

Symbol	**Description**	**Default / Suggested**
α	Fraction of sampled roots allocated to expansion-regime training	> 0.5 (TBD)
λ	Exponential decay for uncertainty targets	5/6 (from KataGo)
d_z	Projection dimension (flat $\mathbf{z}$)	32–64
r	Channel depth (spatial $\mathbf{z}$)	4–8
	Annealing schedule for LoTE/LoTV regularization	TBD

Table 1: System hyperparameters.